

Projet *Bruine* : Analyse V 1.0

1. Table des matières

- 1. Table des matières
- 2. Objectif du document
- 3. Sélection des use cases
- 4. use case
 - 4.1. Groupe 1 : authentification
 - 4.1.1. Se connecter via Steam
 - 4.1.2. Se déconnecter
 - 4.2. Groupe 2 : marché
 - 4.2.1. Vendre une carte
 - 4.2.2. Acheter une carte
 - 4.3. Groupe 3 : système gacha
 - 4.3.1. Effectuer un tirage Gacha
 - 4.3.2. Consulter son inventaire
 - 4.3.3. Convertir des cartes en XP
 - 4.4. Groupe 4 : deck
 - 4.4.1. Gérer son deck
- 5. Regroupement des classes
 - 5.1. Groupe domaine
 - 5.2. Groupe cycle de vie
 - 5.3. Groupe service
 - 5.4. Groupe interface utilisateur et système
- 6. Terminologie

2. Objectif du document

Analyse fondée sur l'expression des besoins du projet **Bruine**. Le but est d'établir la liste des classes métier nécessaires pour modéliser chaque use case, et de voir comment elles seront manipulées par l'application.

3. Sélection des use cases

On évalue selon les critères risques et pertinence, en partant du principe qu'on démarre par ce qui est à haut risque et à grande pertinence.

Pour les risques :

1. j'ai déjà fait ça ;
2. c'est un cas connu, d'autres l'ont fait ;
3. c'est un problème bien documenté ;
4. c'est un problème à creuser, mais des exemples similaires existent ;
5. c'est innovant.

Pour la pertinence :

1. son absence serait à peine remarquée ;
2. le système reste cohérent sans cette fonctionnalité ;
3. la majeure partie du système peut fonctionner ;
4. une partie du système peut fonctionner ;
5. le système ne fonctionnera pas sans cette fonctionnalité.

use case	Risques	Pertinence	Prioritaire
Se connecter via Steam	3	5	oui
Se déconnecter	1	4	oui
Vendre une carte	3	5	oui
Acheter une carte	3	5	oui
Effectuer un tirage Gacha	2	5	oui
Consulter son inventaire	1	4	oui
Convertir des cartes en XP	2	5	oui
Gérer son deck	2	5	oui
Annuler sa mise en vente	1	4	non
Consulter un profil Steam	3	3	non
Consulter le classement	1	3	non
Supprimer son compte Bruine	1	2	non
Gérer les comptes admin (CRUD)	1	3	non
Gérer les utilisateurs Steam	1	3	non
Gérer les récompenses Gacha (CRUD)	1	3	non

Les use cases non prioritaires sont soit des opérations CRUD triviales (administration), soit des vues en lecture seule sans logique métier complexe.

Le cas **Se connecter via Steam** est traité en premier, c'est la porte d'entrée de toutes les fonctionnalités, et l'intégration par OpenID 2.0 avec Steam est une nouvelle notion pour moi.

Les cas **Vendre** et **Acheter une carte** (risque 3) sont traités avant le gacha, malgré la dépendance logique apparente (il faut des cartes pour les vendre). En analyse, l'ordre de traitement suit le risque, pas les dépendances fonctionnelles. Ce sont les use cases qui introduisent les contraintes les plus structurantes sur le modèle.

4. use case

4.1. Groupe 1 : authentication

4.1.1. Se connecter via Steam

Un visiteur clique sur "Se connecter avec Steam". Il est redirigé vers Steam qui vérifie son identité (OpenID 2.0) et renvoie un callback avec son `steamId`. Si c'est sa première connexion, un compte Bruine est créé automatiquement.

Classes candidates

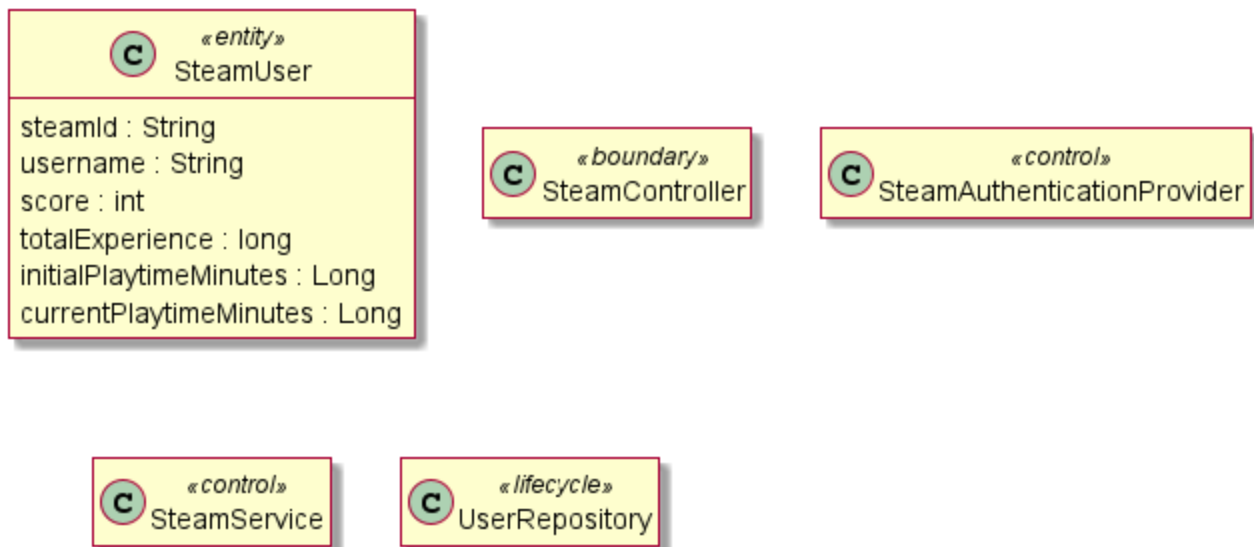
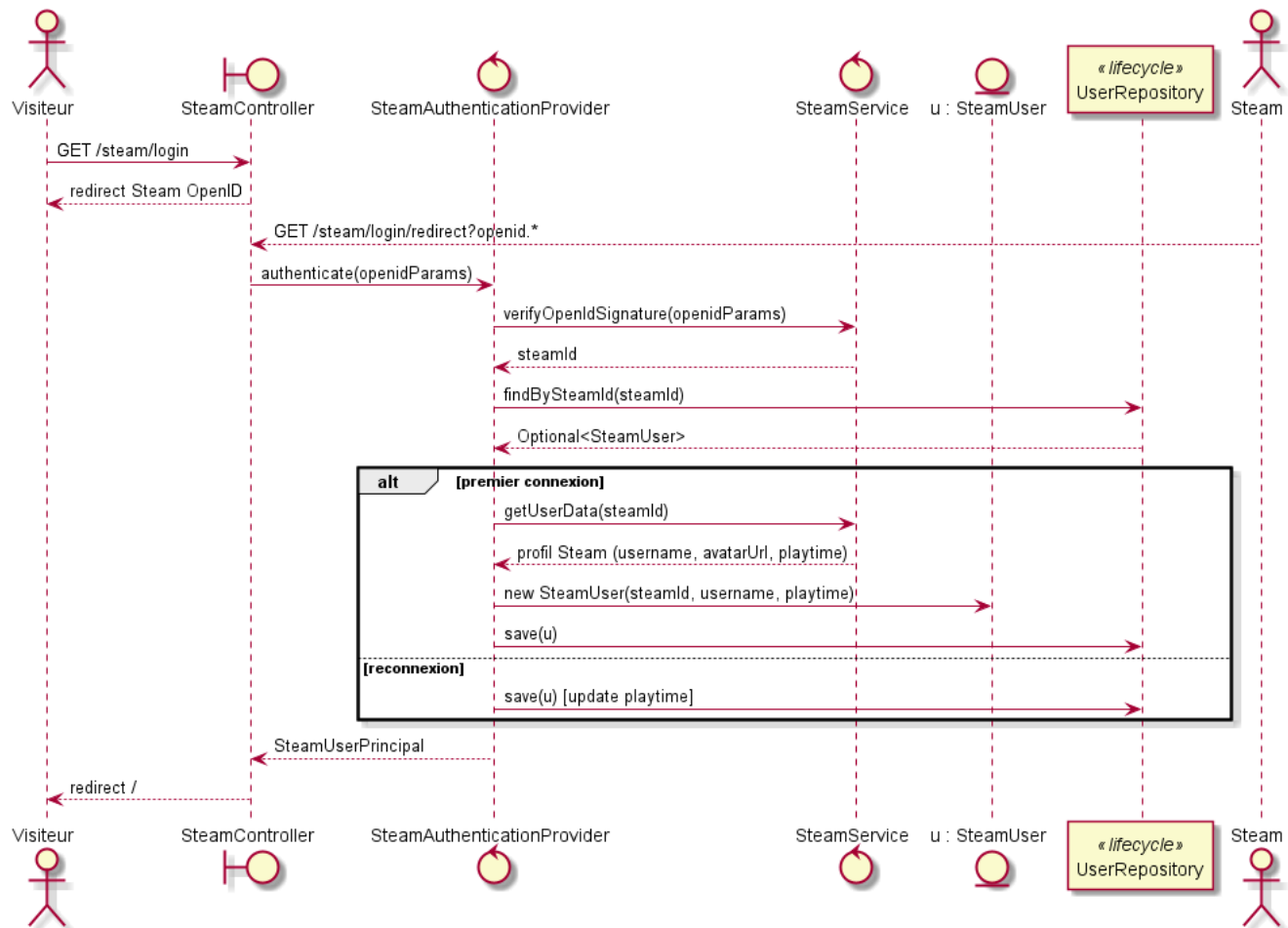
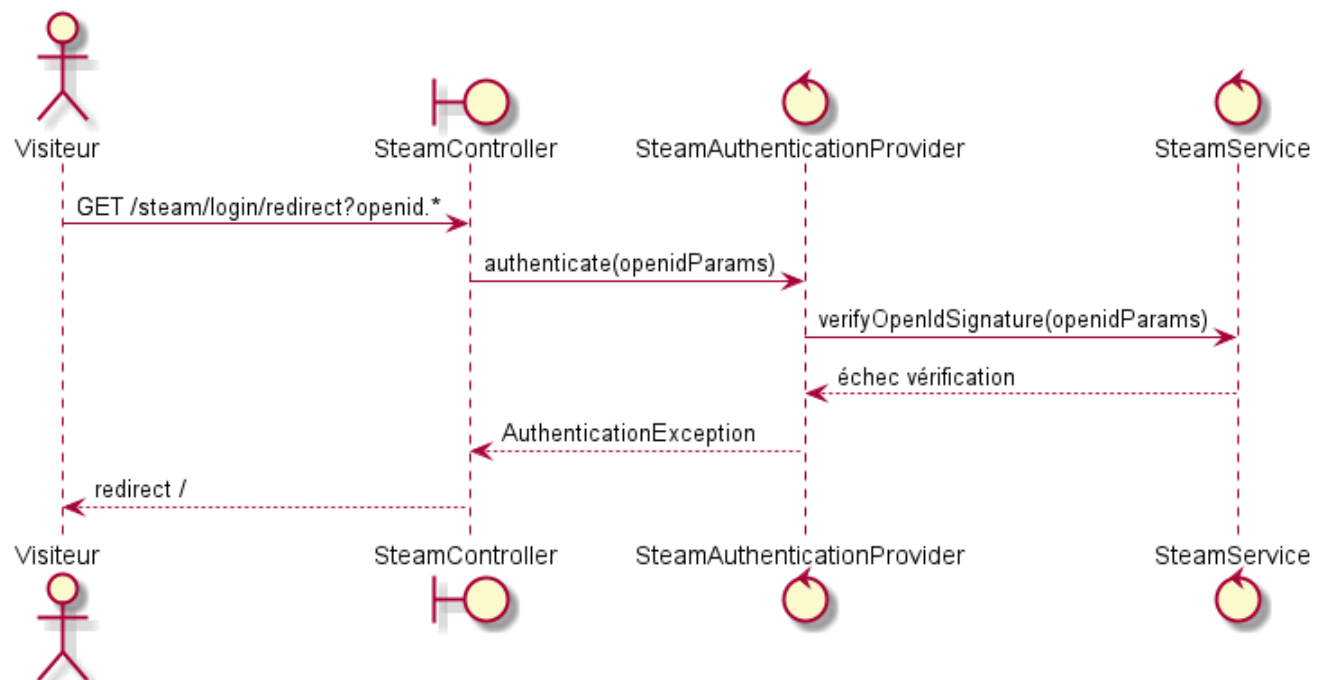


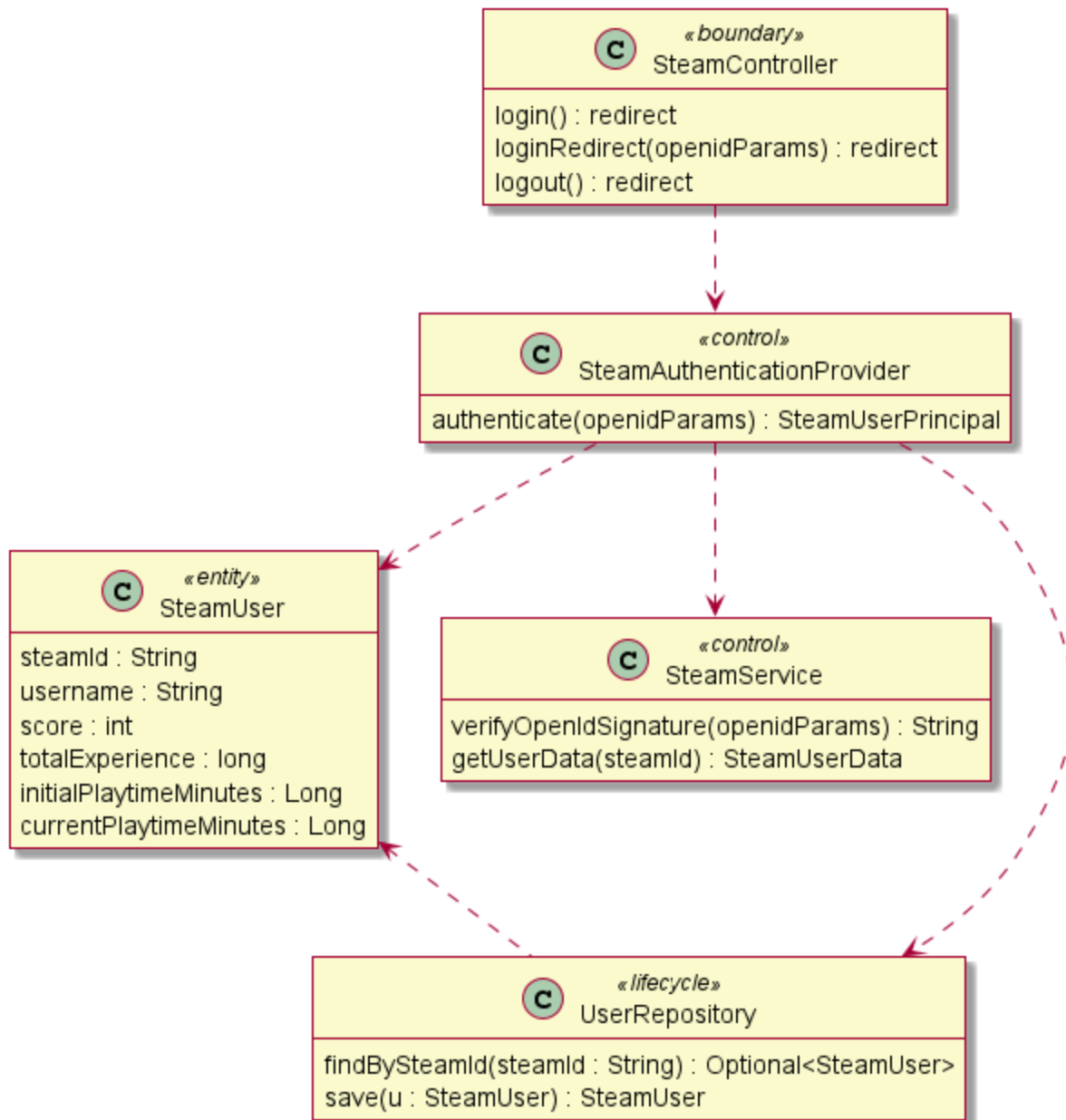
Diagramme de séquence, cas nominal



Déroulement alternatif : signature OpenID invalide



Classes consolidées



4.1.2. Se déconnecter

Un utilisateur Steam ferme sa session sans affecter une éventuelle session admin ouverte simultanément dans le même navigateur.

La contrainte de coexistence des deux sessions dans la même session HTTP rend ce use case non trivial. Invalider la session HTTP globale détruirait les deux contextes de sécurité.

Classes candidates

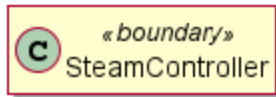
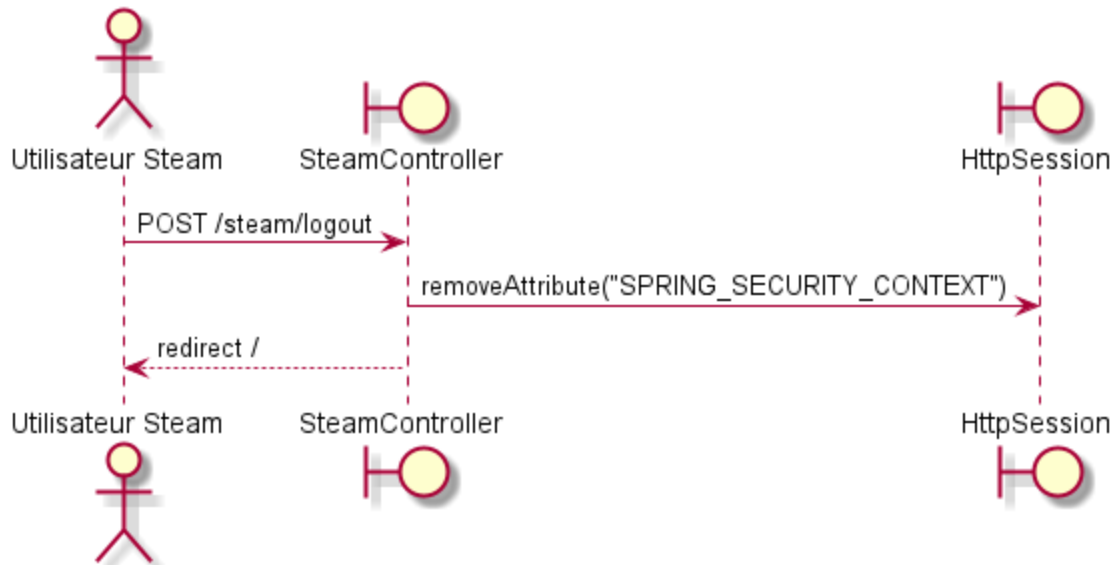
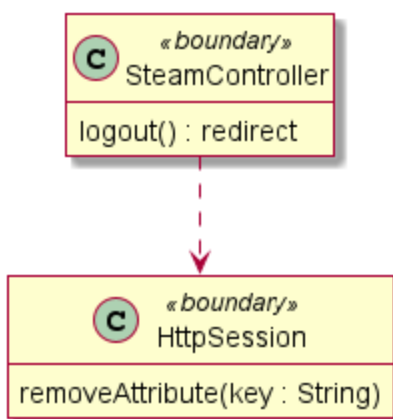


Diagramme de séquence



Remarque : on n'appelle délibérément pas `session.invalidate()` . Cette décision découle de la contrainte d'isolation des sessions. Elle sera documentée tout au long du développement.

Classes consolidées



4.2. Groupe 2 : marché

Le marché est traité avant le gacha car il introduit les contraintes les plus structurantes sur le modèle :

`UserGachaReward` doit supporter un transfert de propriété, et la protection deck <-> marché impose une dépendance

inter-modules dès la conception du modèle.

4.2.1. Vendre une carte

Le joueur met en vente une carte de son inventaire à un prix en points Bruine. Deux protections s'appliquent, la carte doit lui appartenir et ne doit pas être présente dans son deck. Si les conditions sont remplies, une `MarketListing` est créée.

Classes candidates

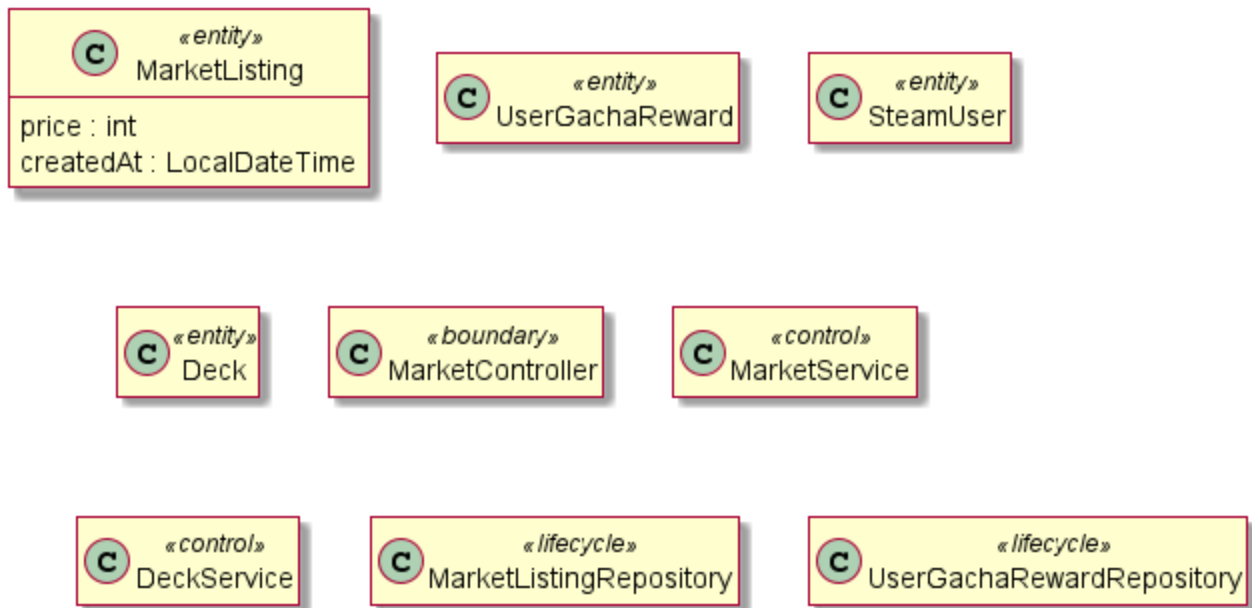
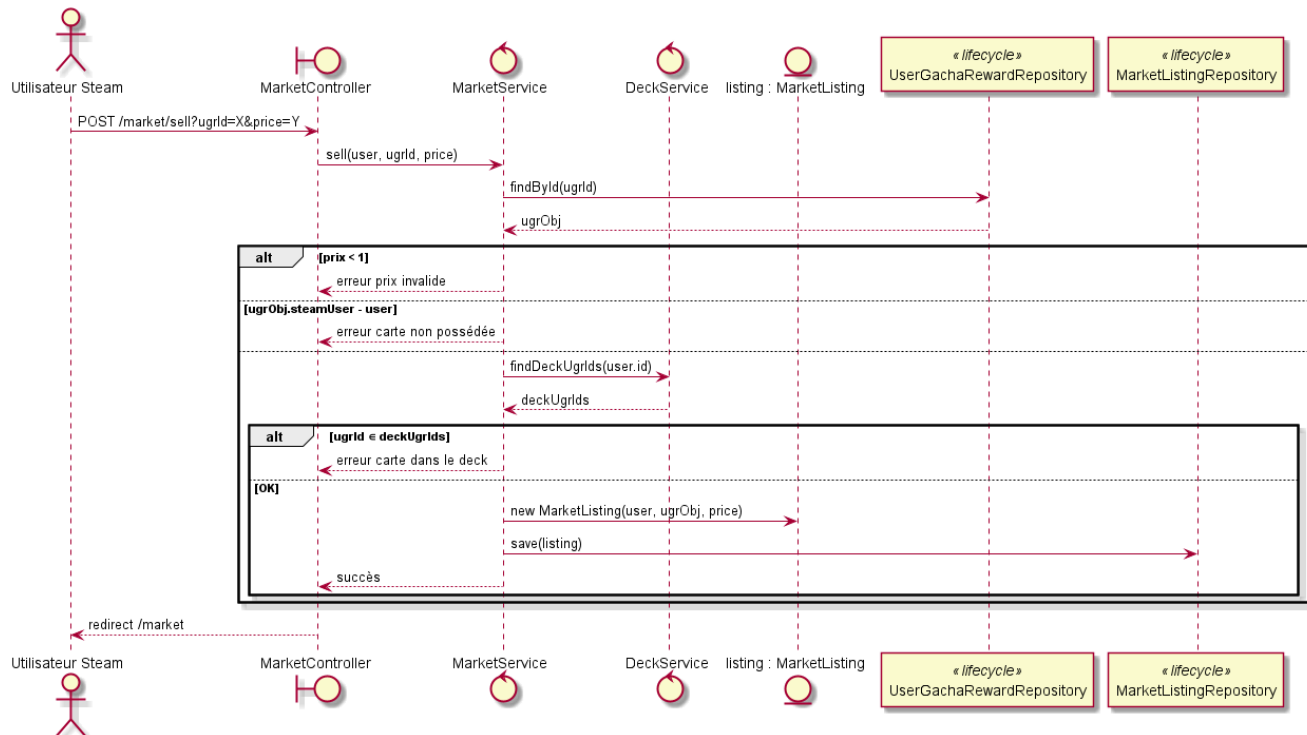
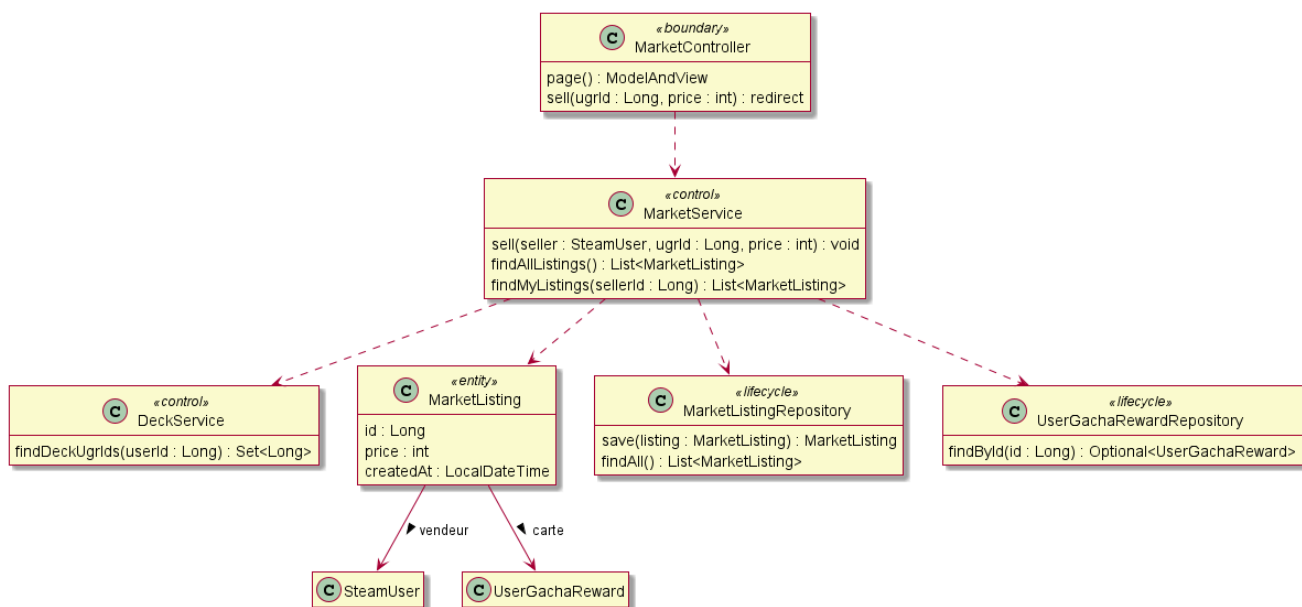


Diagramme de séquence, cas nominal



Classes consolidées



4.2.2. Acheter une carte

L'acheteur sélectionne une annonce disponible. Le système vérifie qu'il n'achète pas sa propre annonce et qu'il dispose du score suffisant. Le score est transféré du vendeur vers l'acheteur, et la propriété de la `UserGachaReward` est transférée. L'annonce est ensuite supprimée.

Classes candidates

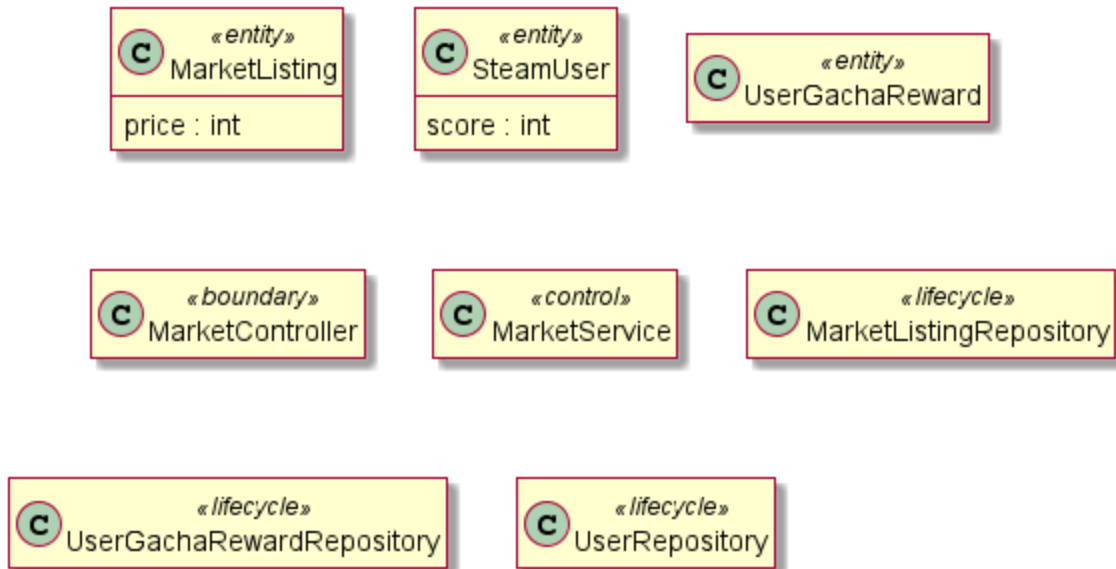
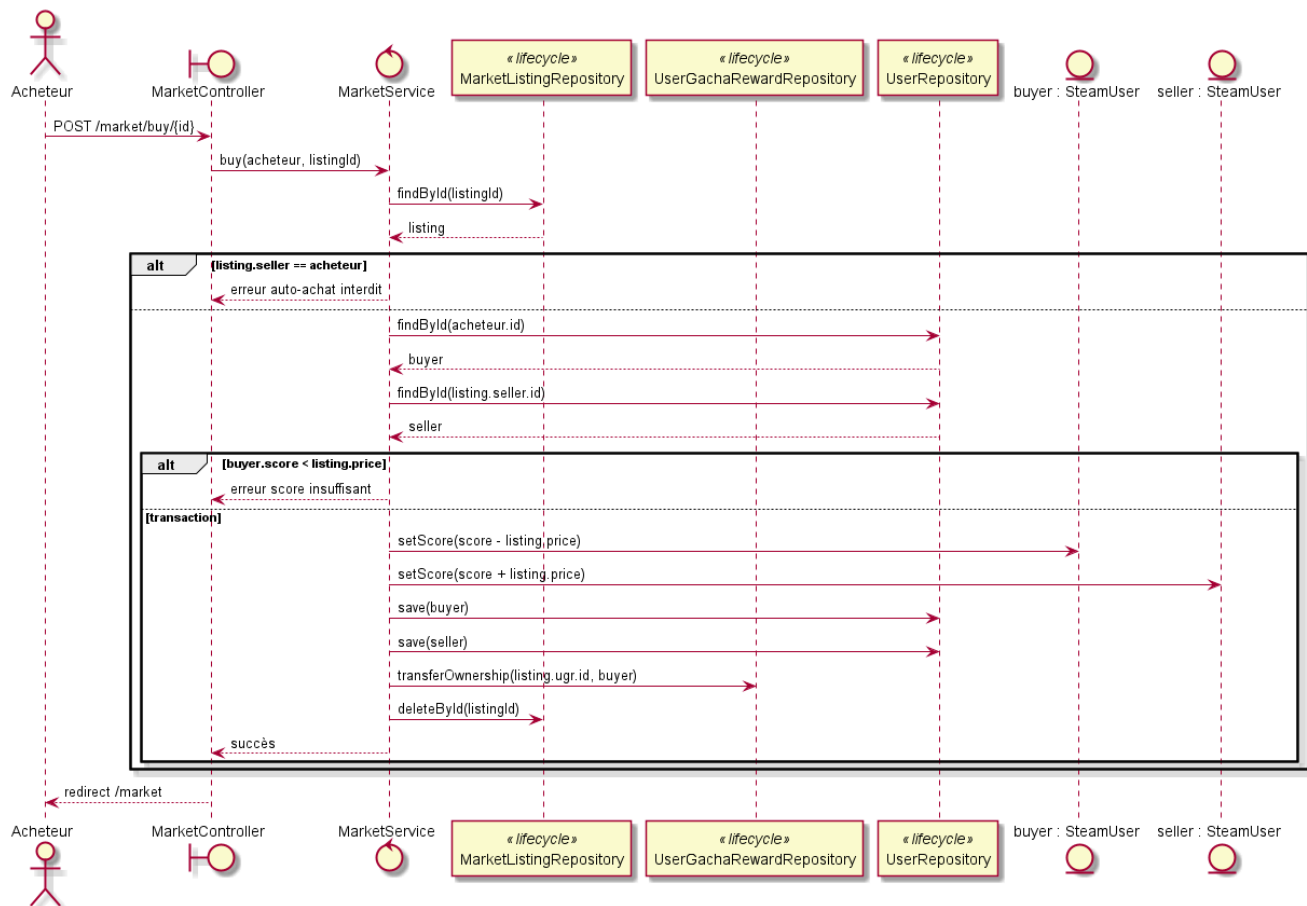
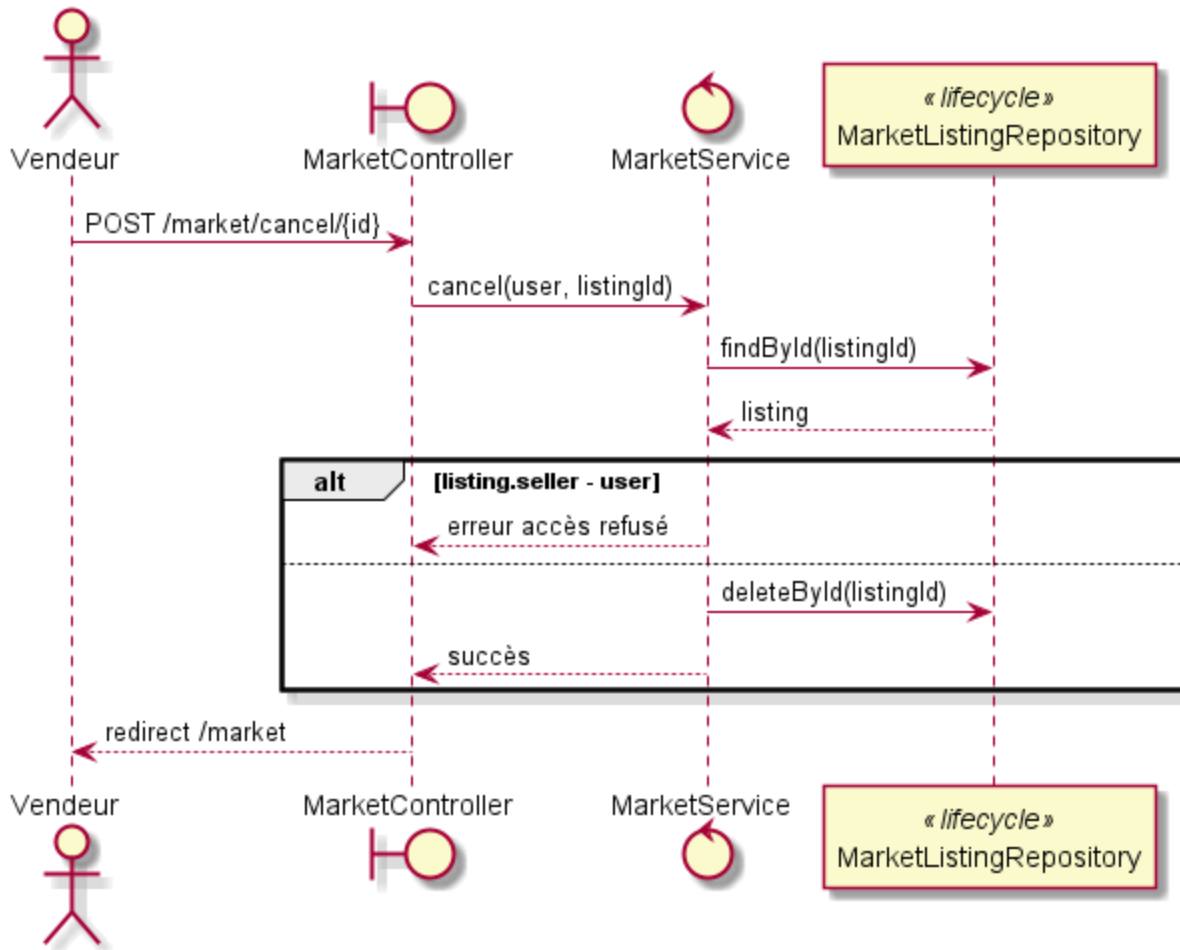


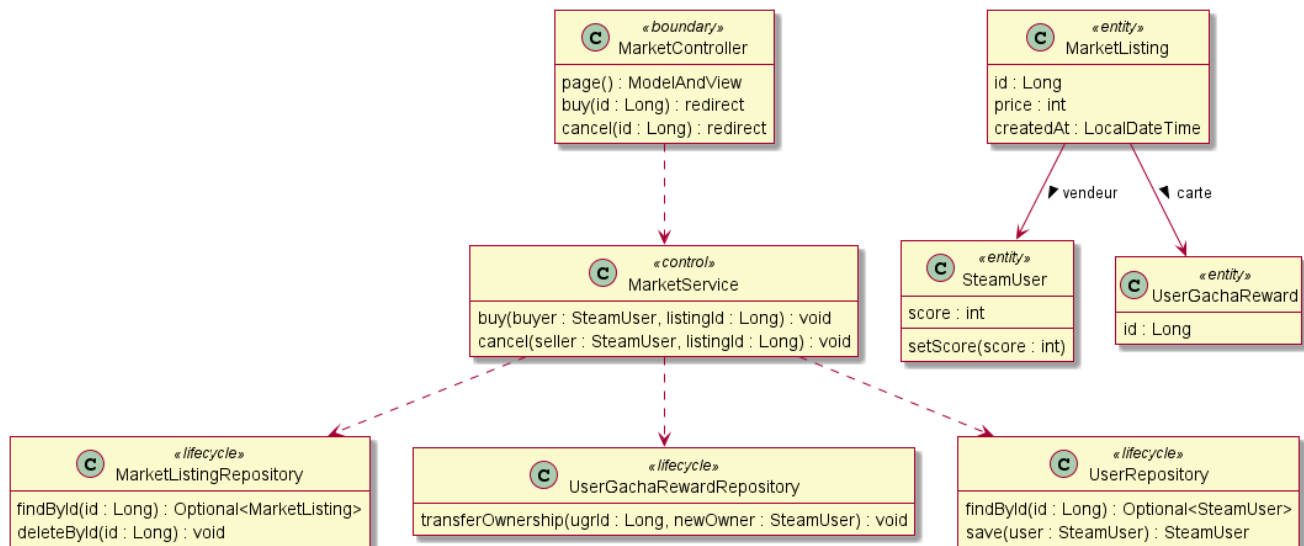
Diagramme de séquence, cas nominal



Déroulement alternatif : annuler sa propre annonce



Classes consolidées



4.3. Groupe 3 : système gacha

4.3.1. Effectuer un tirage Gacha

Un utilisateur authentifié dépense 10 points par tirage (1 à 10 tirages par requête). Le système tire aléatoirement une rareté puis une finition selon des probabilités configurées, puis crée une `UserGachaReward` par carte obtenue.

Classes candidates

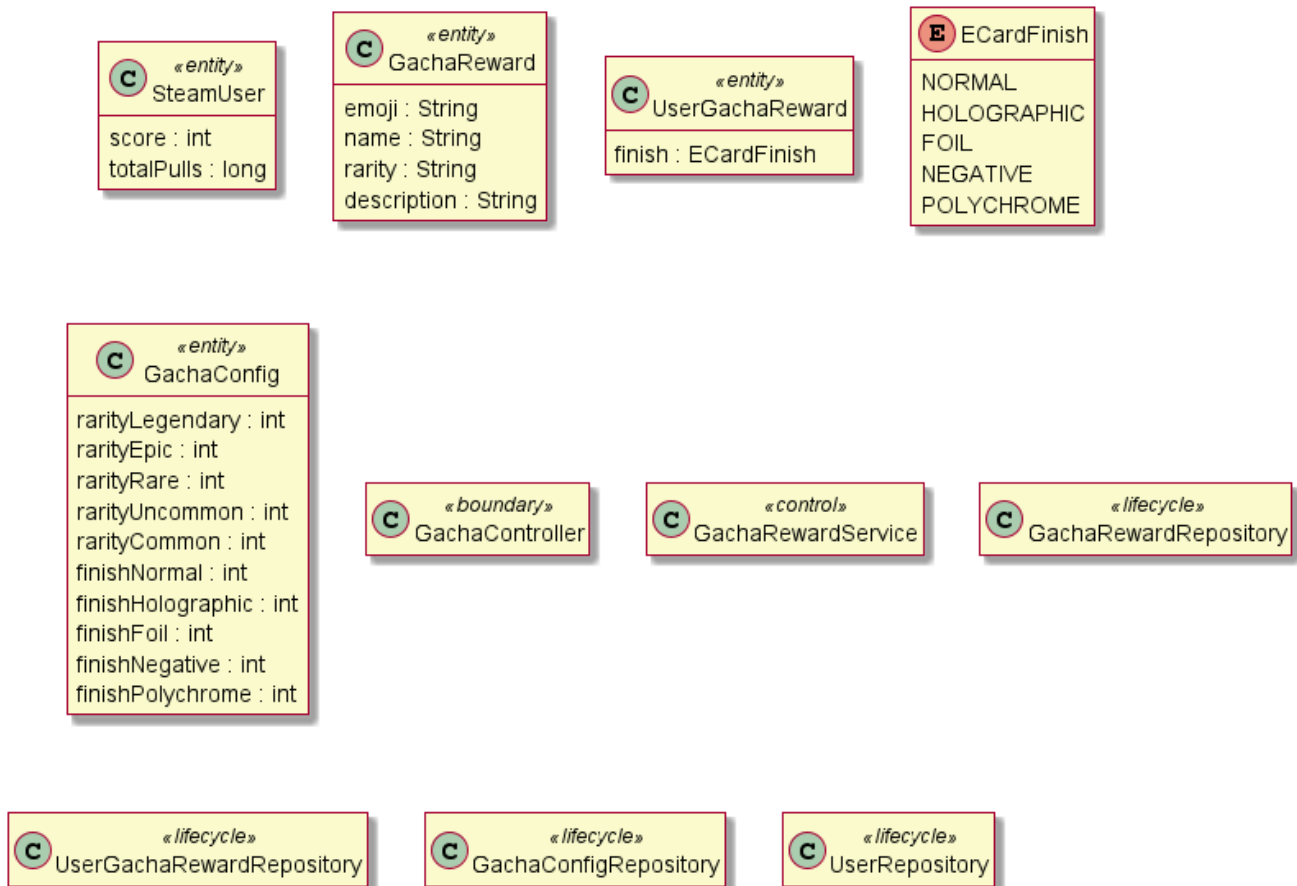
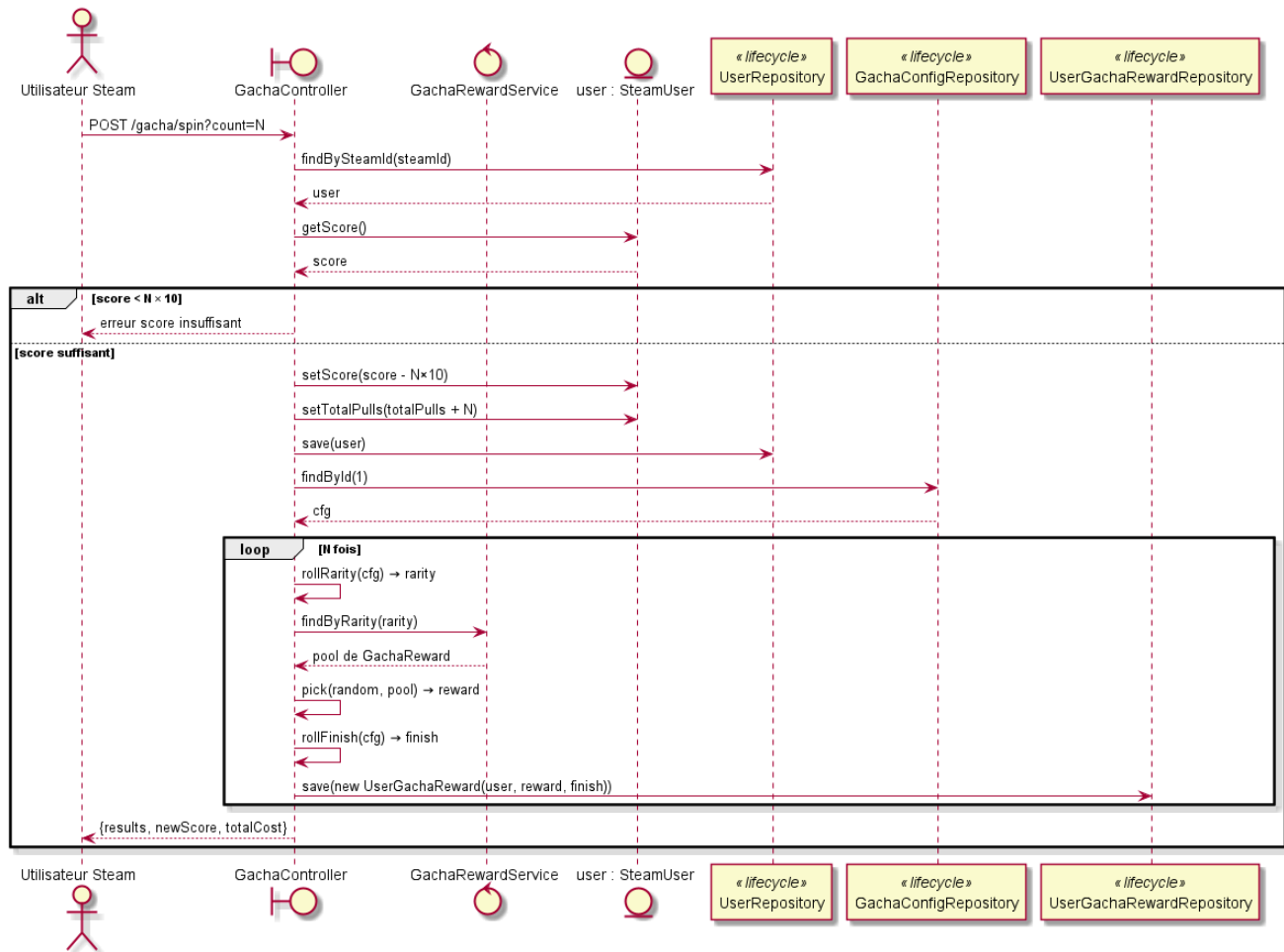
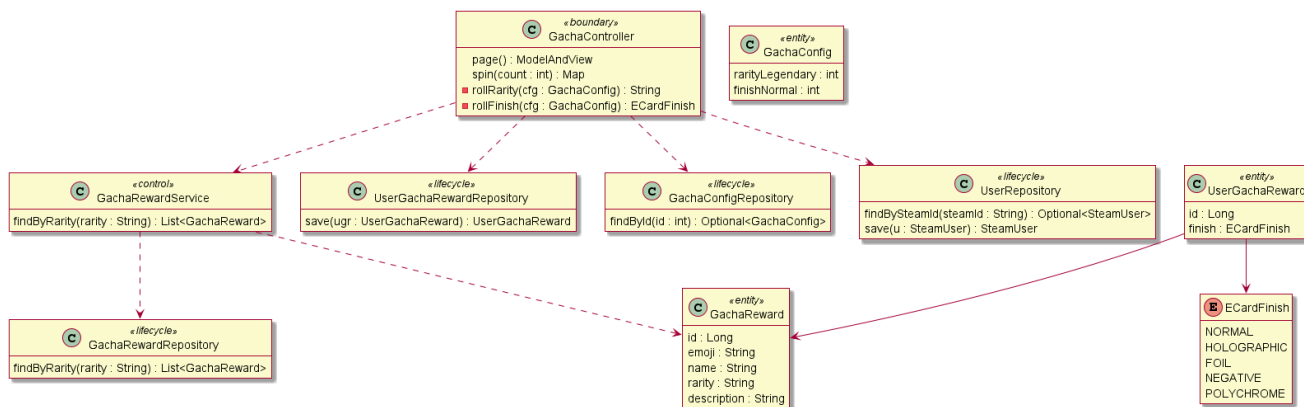


Diagramme de séquence, cas nominal



Classes consolidées



4.3.2. Consulter son inventaire

L'utilisateur accède à la liste de toutes ses cartes, triées par rareté décroissante. Les cartes actuellement listées sur le marché sont exclues de l'affichage.

Ce use case est essentiellement une lecture mais avec une particularité avec un filtrage sur les cartes en vente, l'inventaire ne doit montrer que les cartes disponibles, pas celles engagées sur le marché.

Classes candidates

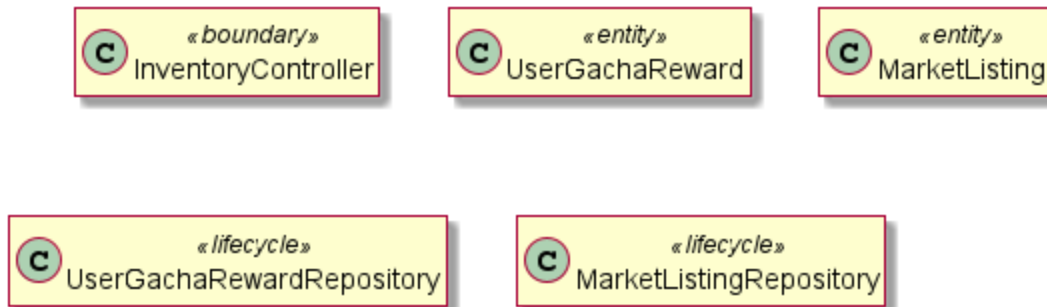
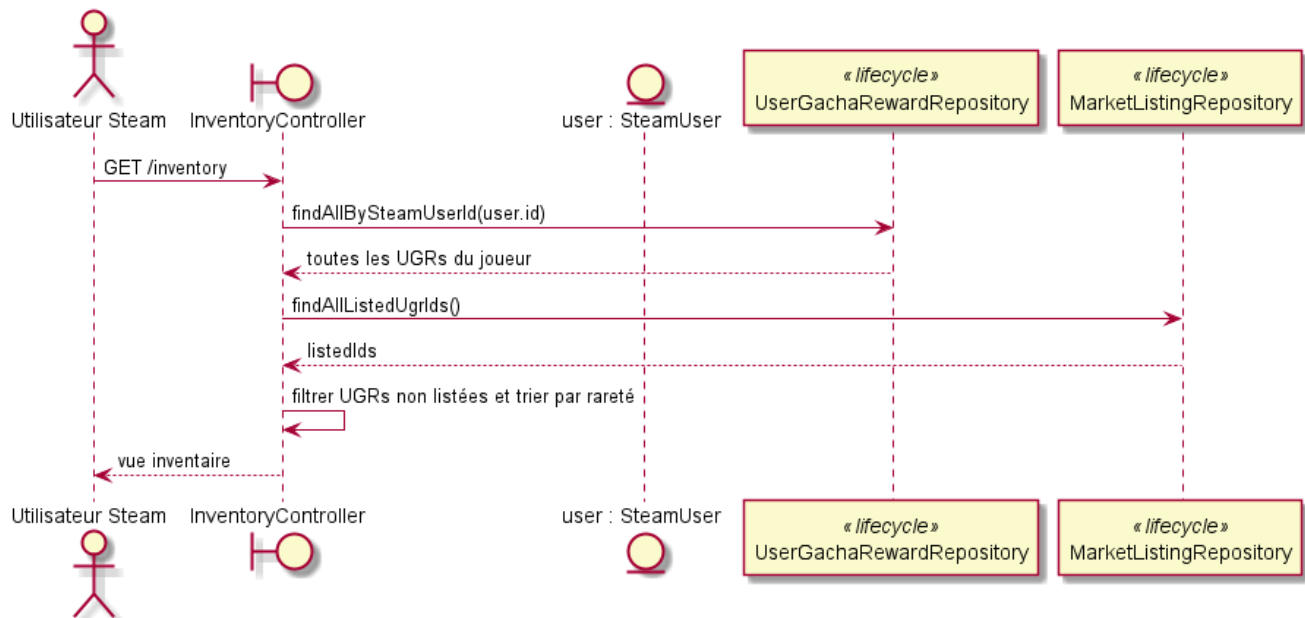
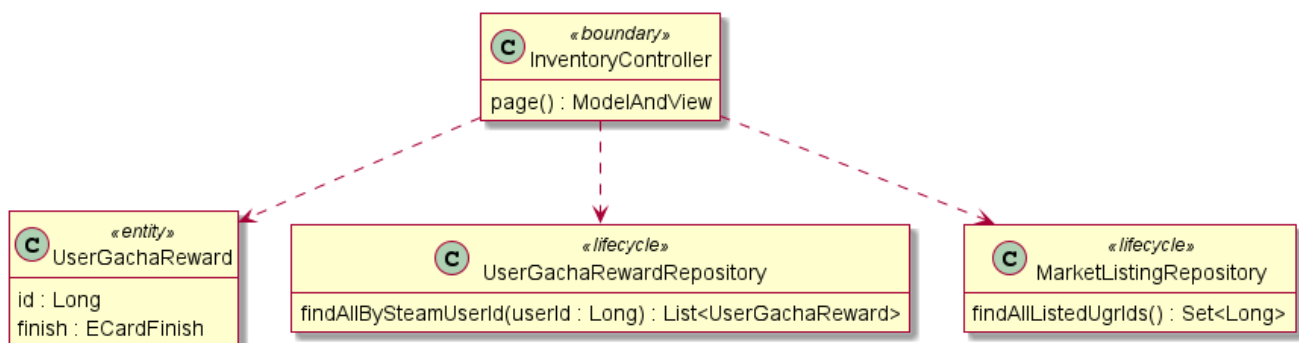


Diagramme de séquence



Classes consolidées



4.3.3. Convertir des cartes en XP

L'utilisateur sélectionne des cartes via drag & drop dans la " Steam Machine " et confirme la conversion. Chaque carte est détruite et une quantité d'XP calculée selon la formule $XP = base_rareté \times multiplicateur_finish$ est créditée.

Les cartes présentes dans le deck ou en vente sur le marché sont automatiquement exclues.

Classes candidates

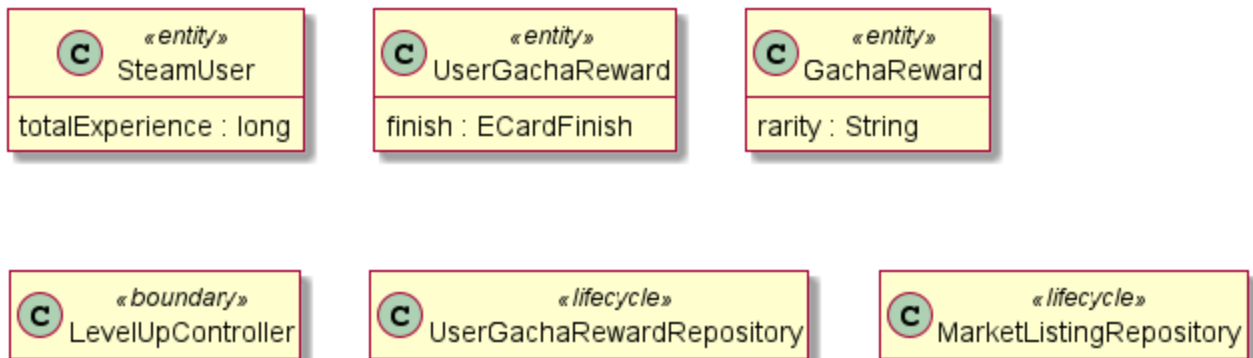
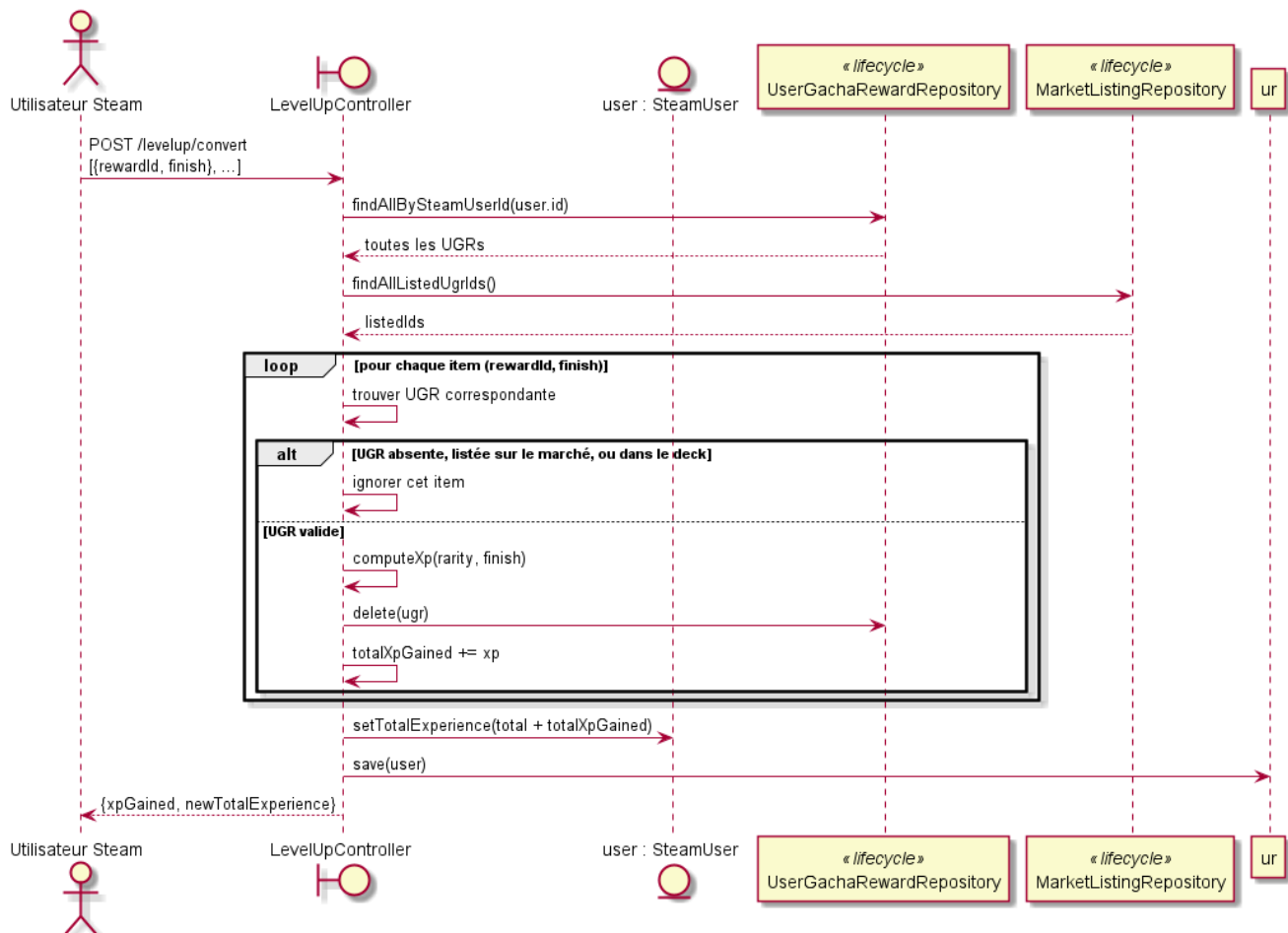


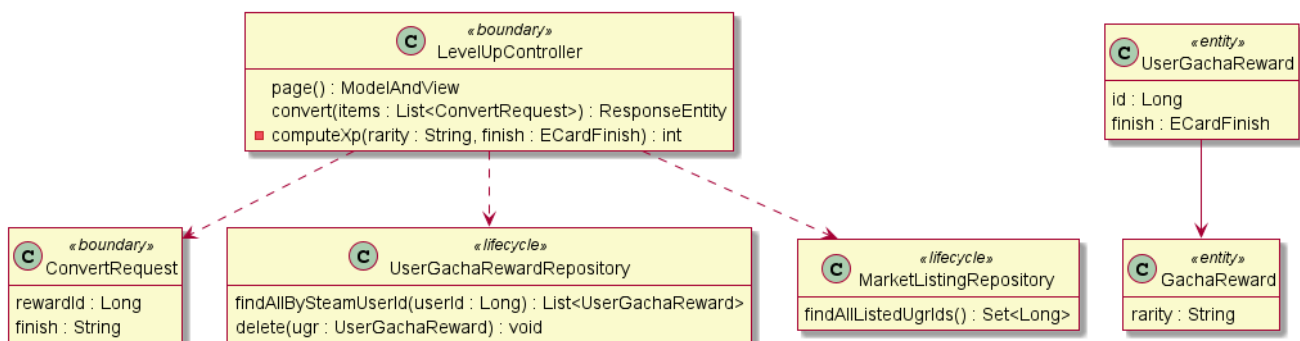
Diagramme de séquence, cas nominal



Remarque : le calcul de l'XP (de base) est le suivant :

Rareté	Base XP	Finish	Multiplicateur
Common	10	NORMAL	×1
Uncommon	25	HOLOGRAPHIC	×2
Rare	75	FOIL	×3
Epic	200	NEGATIVE	×4
Legendary	500	POLYCHROME	×5

Classes consolidées



4.4. Groupe 4 : deck

4.4.1. Gérer son deck

L'utilisateur compose un deck de 0 à 10 cartes via drag & drop. Il peut mettre la même carte plusieurs fois (doublons autorisés) et plusieurs finitions d'une même carte. La sauvegarde remplace le deck existant en entier.

Les cartes actuellement en vente sur le marché ne peuvent pas être ajoutées au deck.

Classes candidates

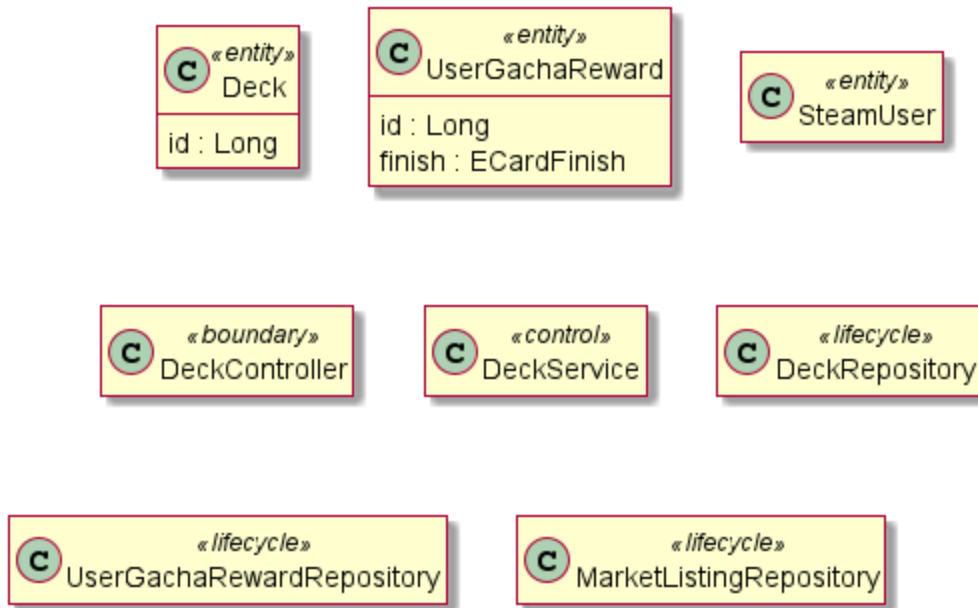


Diagramme de séquence : affichage de la page

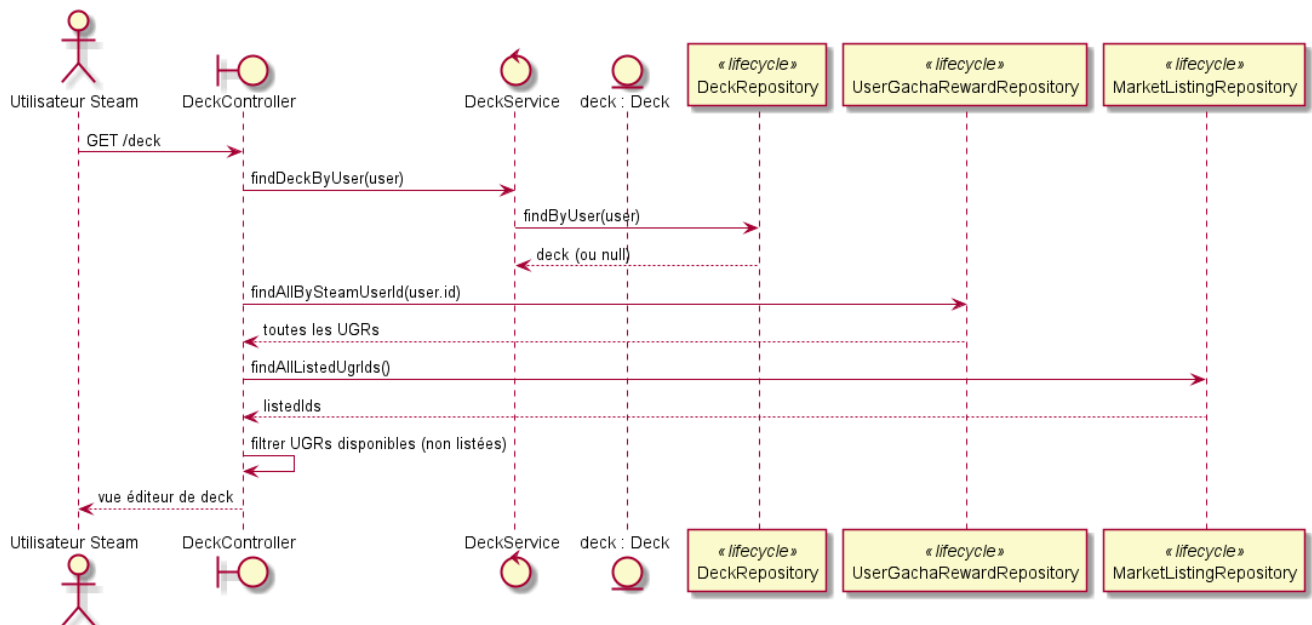
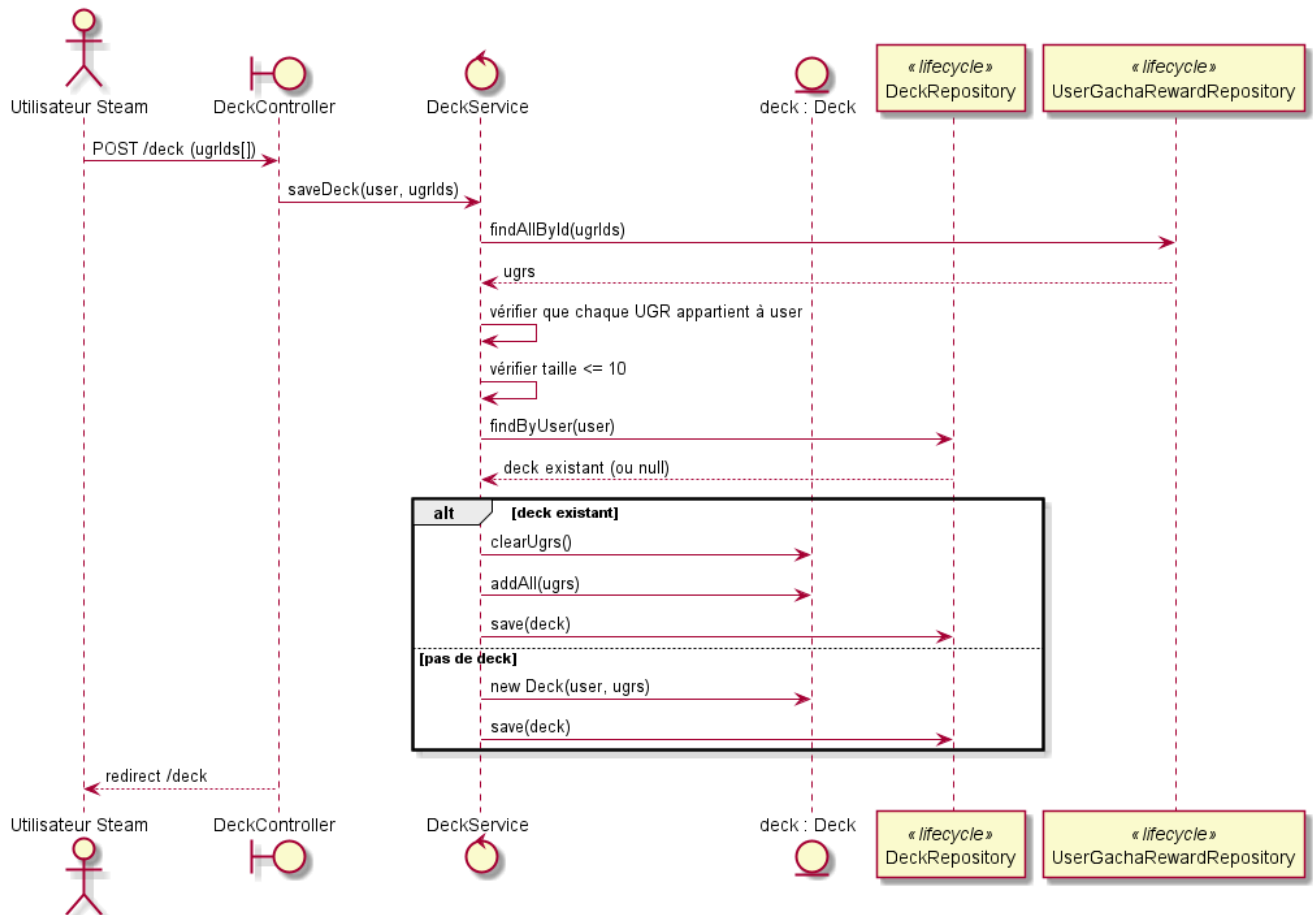
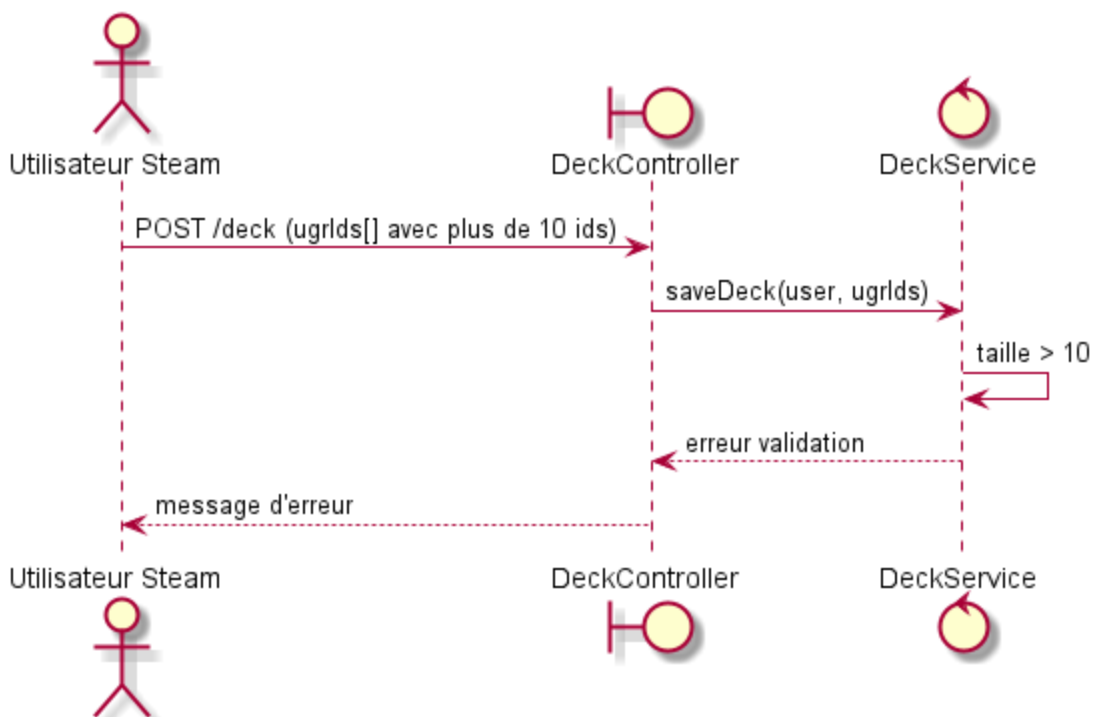


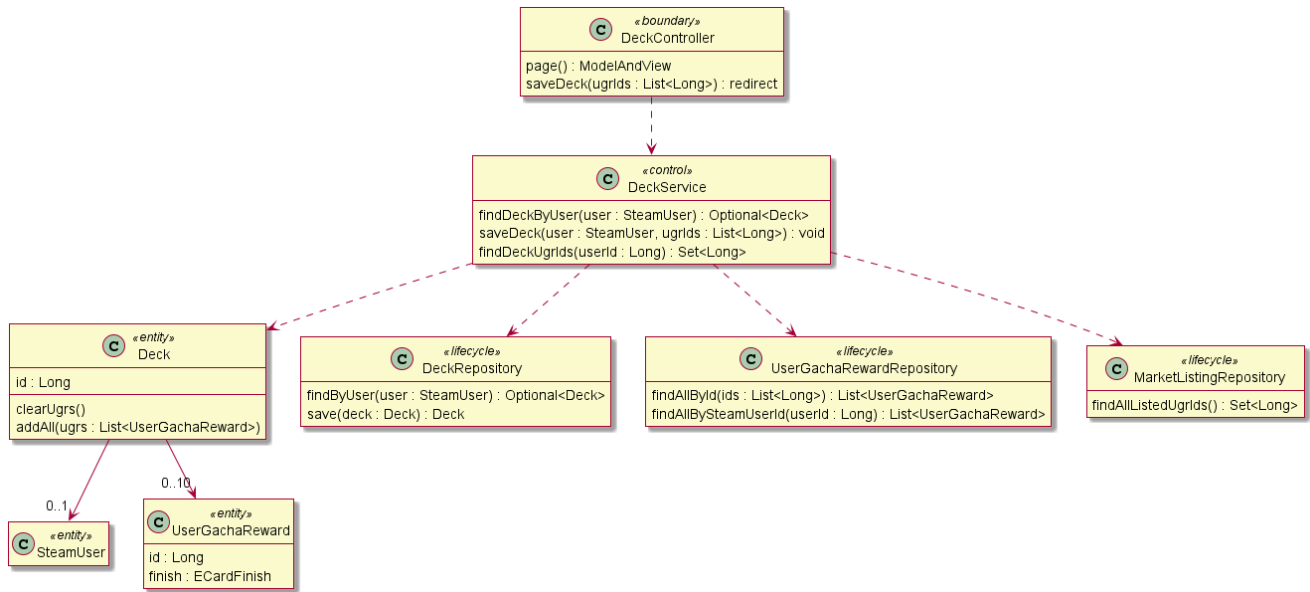
Diagramme de séquence : sauvegarder le deck



Déroulement alternatif : plus de 10 cartes



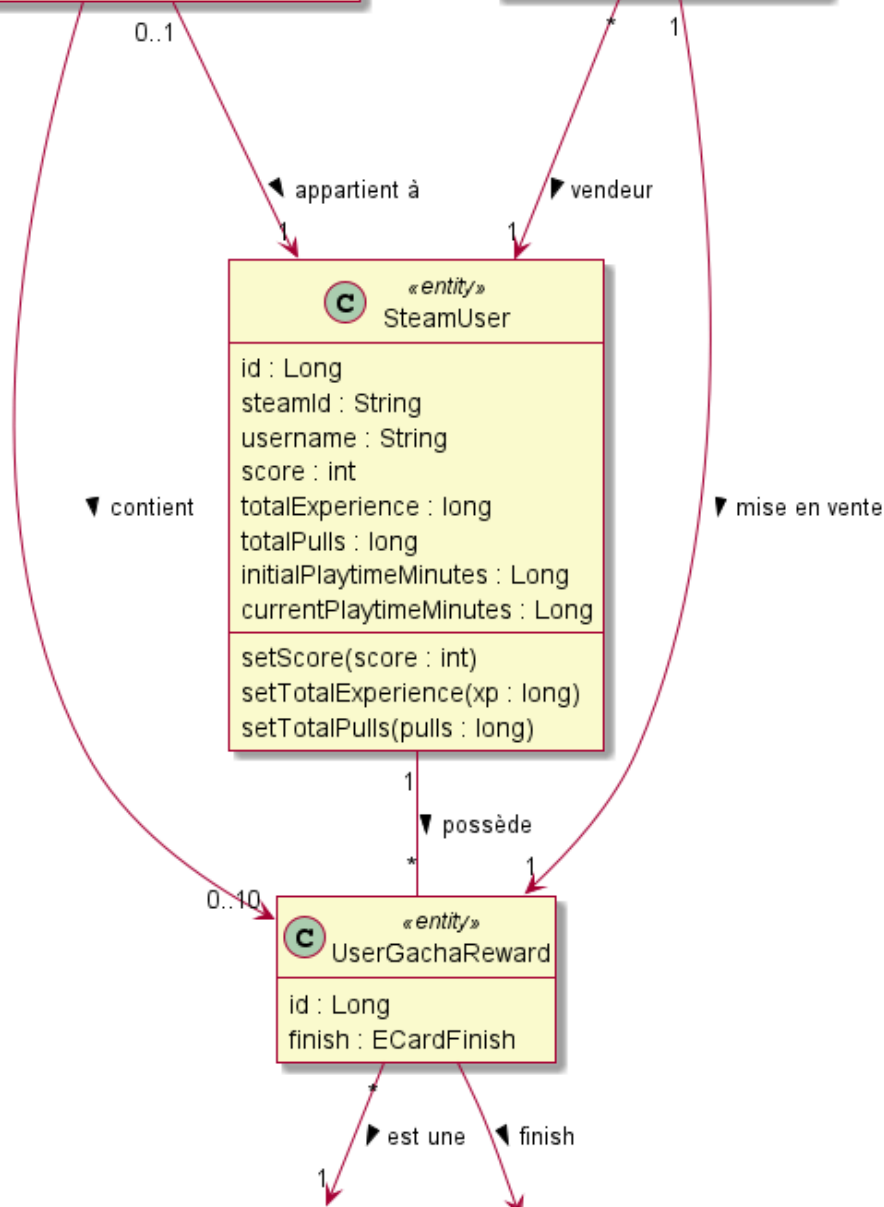
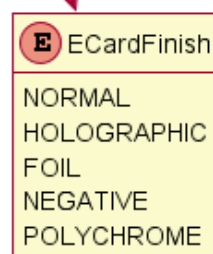
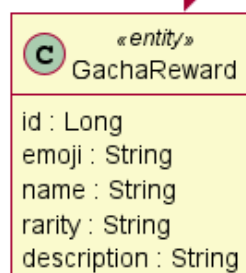
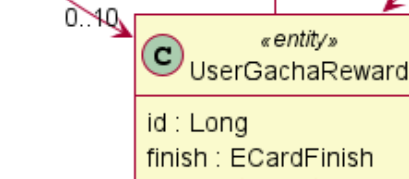
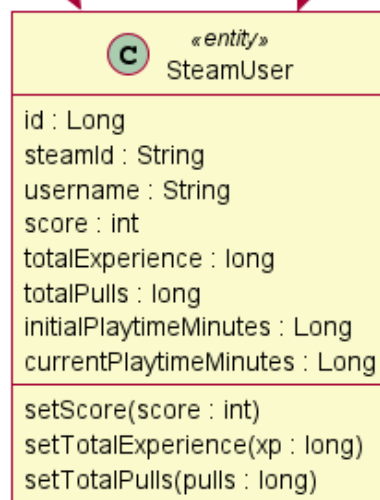
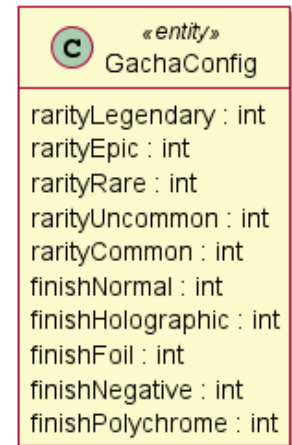
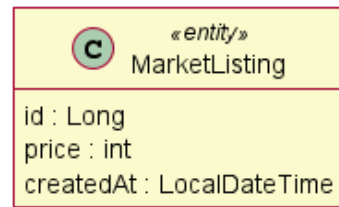
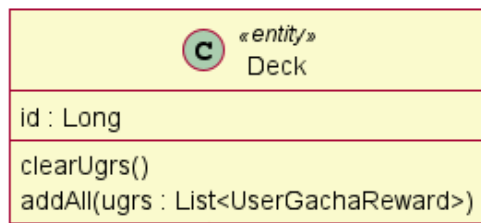
Classes consolidées



5. Regroupement des classes

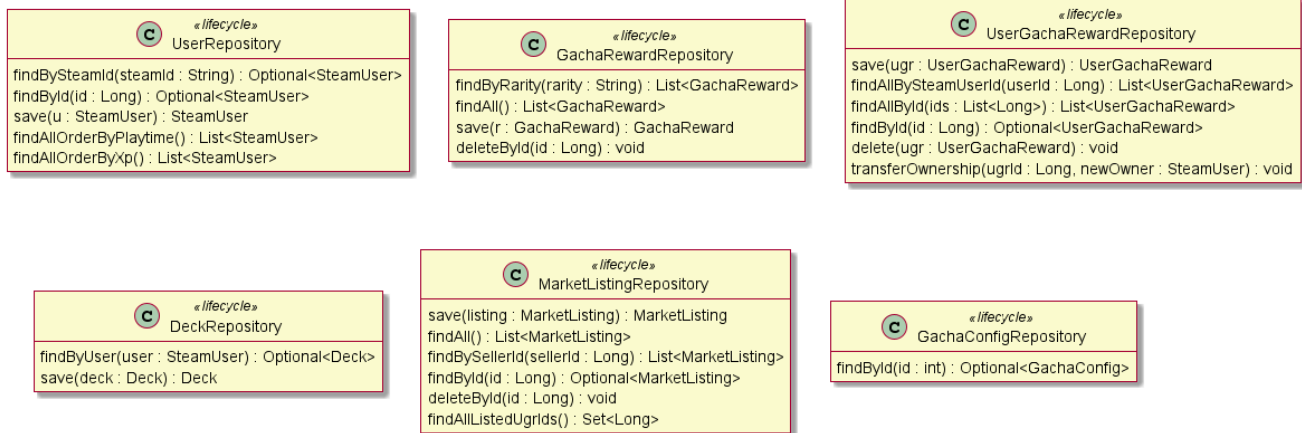
5.1. Groupe domaine

Ce groupe contient les entités métier du domaine Bruine. `SteamUser` est l'entité centrale ; toutes les autres y sont reliées directement ou indirectement.

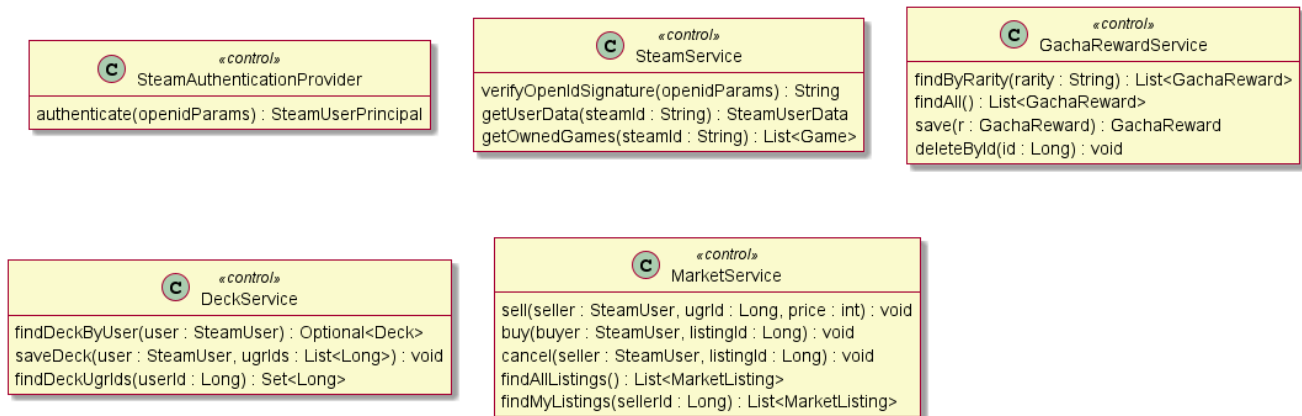


Remarque : UserGachaReward est l'entité pivot de l'application. Elle est produite par le gacha, consommée par le level up, référencée par le deck, et mise en vente sur le marché. Chaque copie physique d'une carte est une ligne distincte, ce qui permet de la référencer individuellement dans toutes ces opérations.

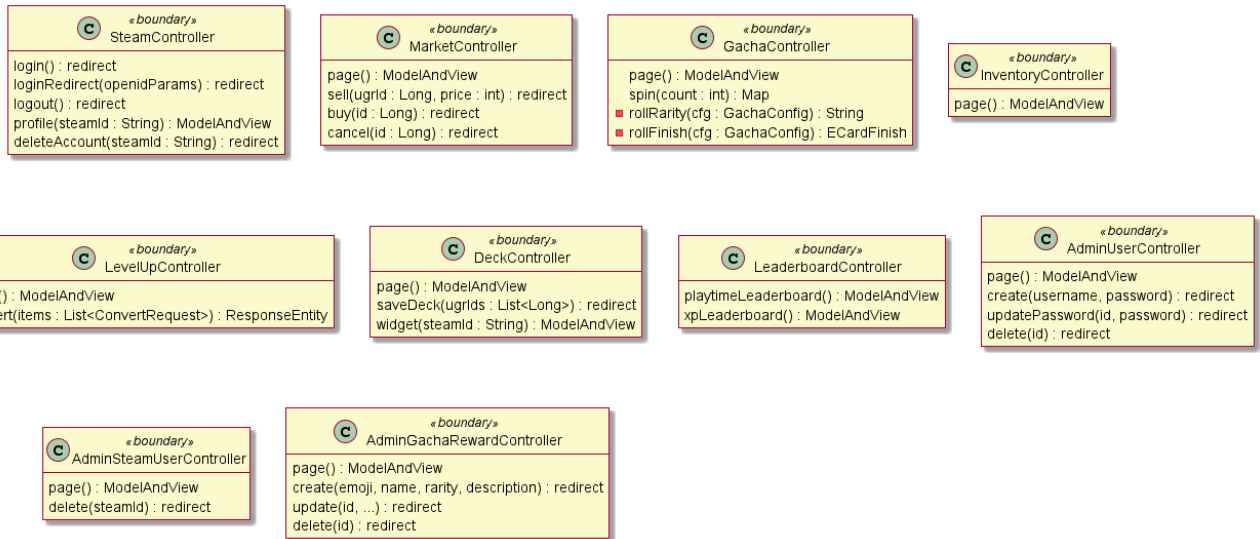
5.2. Groupe cycle de vie



5.3. Groupe service



5.4. Groupe interface utilisateur et système



6. Terminologie

Terme	Définition
UGR/ugr	UserGachaReward : copie physique d'une carte appartenant à un joueur. Chaque tirage crée une ligne distincte.
Score	Monnaie interne de Bruine (💧), gagnée via le temps de jeu Steam. Utilisée pour les tirages et le marché.
Finish	Finition d'une carte (effet visuel) : Normal, Holographique, Foil, Négatif, Polychrome.
Rareté	Niveau de rareté d'une carte : Common, Uncommon, Rare, Epic, Legendary.
Deck	Sélection de 0 à 10 cartes exposées sur le profil public du joueur via un widget.
Listing	Annonce de vente d'une carte sur le marché (<code>MarketListing</code>).
XP	Expérience cumulée via la conversion de cartes (<code>totalExperience</code>).
steamId	Identifiant unique Steam au format 7656119xxxxxxxxx (17 chiffres).
OpenID 2.0	Protocole d'authentification délégué utilisé par Steam. Bruine ne manipule aucun mot de passe Steam.
Boundary	Classe d'interface entre le système et un acteur (contrôleur HTTP, vue, API externe).
Control	Classe de coordination métier (service, workflow). Porte la logique, délègue la persistance.
Lifecycle	Classe responsable du cycle de vie des entités en base de données (repository).